

Ionixx Backend Developer Technical Challenge

CONFIDENTIAL – Do not distribute

Expectation from Candidates for the Assessment

We expect the given assignment to be completed with production-grade quality. The submission should not only contain working code but should also demonstrate good coding standards, proper structure, and clean implementation.

During evaluation, we will look at:

- The concepts and design approach used to solve the problem
- Code quality and coding standards
- Unit test cases provided
- Coverage of edge cases and error handling

You are free to use AI assistance or tools for code generation. However, we expect you to have a clear understanding of the code you submit, as we will assess your final implementation and your ability to explain the approach.

Ultimately, we are evaluating what your final production-ready solution looks like.

Overview

This take home challenge is intended to help us understand how you solve technical problems, similar to the types of problems we work on.

Some of the requirements may seem ambiguous - they're intentionally designed that way. We want to be able to evaluate both your technical skills and how you navigate ambiguity. Feel free to expand on this problem and document your assumptions.

The challenge should be completed in NodeJs with Typescript. While the programming language is fixed, you may use any library or framework. We recognize that writing good

software doesn't mean writing things from scratch, so treat this challenge just like any other project you would work on.

Please use LLMs as you see fit. LLMs are part of a modern engineer's toolkit and we understand that.

Problem Description

We are working with a robo-advisor who is interested in using our developer APIs to automate their managed investments (model portfolios) offering.

For a proof-of-concept, we've promised to deliver:

- An order splitter where they would pass us a user's model portfolio and the amount as input to the order splitter, and we'd send them back: ○ The amount and quantity of stocks they would need to place orders for as output from the order splitter, based on the model portfolio.
- When the orders should be executed (markets are open Monday through Friday).
- Historic orders

Example:

- A partner's user wants to invest USD 100 in a model portfolio containing 2 stocks: AAPL and TSLA, which make up 60% and 40% of the model portfolio respectively.
- The order splitter should respond back with the amount (i.e. \$60 of AAPL and \$40 of TSLA) and the corresponding number of shares to be purchased for each stock based on the stock price at time of purchase.

Requirements

Functional Requirements

- Develop an API endpoint or endpoints that:

- Accepts the model portfolio and the total amount to be purchased/sold as input.
- Responds back with a breakdown of the amount and number of shares to purchase/sell for each stock in the model portfolio and when to execute the orders.
- Returns historic orders.
- Internally, we should be able to configure the number of decimal places allowed for the quantity of shares to be purchased/sold (e.g. we may accept buy/sell orders for shares up to 3 decimal places today, but may change that to allow up to 7 decimal places in future).
- For simplicity, we'll fix the price of any stock to \$100. However, if the partner decides to pass the market price for the stocks within the model portfolio, it should take priority over the fixed price we have defined.

Technical Requirements

- The API design must follow RESTful conventions.
- You have to define the endpoints and specify how you would take in data (i.e. format, data type, etc.).
- Performance is important. Instrument the response times in milliseconds so that this is visible in the console.
- The endpoint should be flexible in its design. It should allow for different model portfolios and different order types (BUY or SELL).
- Data should not survive application restart.

Submission

Submit a source code in your github profile and make the repo as publicly available.

Your submitted source code:

- Must compile.
- Should include a README containing:
 - Execution instructions with request examples
 - Documentation of libraries or framework dependencies used
- Should include an ANSWERS file containing responses to below questions:

- What was your approach (thought process) to tackling this project?
- What assumptions did you make?
- What challenges did you face when creating your solution?
- If you were to migrate your code from its current standalone format to a fully functional production environment, what are some changes and controls you would put in place (e.g. security controls)?
- If you've used LLMs to solve the challenge, describe how and where you've used it and how did it help you in tackling the challenge? Provide specific examples and details.